# Dynamics of Hallucination Propagation in Multi-Agent LLM Collaboration on Graph Topologies with Penalty-Based Enforcement

**Alan Jacob**
*School of Computing*
*Newcastle University*
Newcastle, UK
A.Jacob2@newcastle.ac.uk

**Apoorv Mani**
*School of Computing*
*Newcastle University*
Newcastle, UK
A.Mani3@newcastle.ac.uk

**Chris George**
*School of Computing*
*Newcastle University*
Newcastle, UK
C.J.George2@newcastle.ac.uk

**Ramya Varunsegar**
*School of Computing*
*Newcastle University*
Newcastle, UK
R.P.Varunsegar2@newcastle.ac.uk

**Shreya Patil**
*School of Computing*
*Newcastle University*
Newcastle, UK
S.S.Patil3@newcastle.ac.uk

**Shreyas Shetty**
*School of Computing*
*Newcastle University*
Newcastle, UK
S.R.K.Shetty2@newcastle.ac.uk

***Abstract*—Large Language Models (LLMs) have demonstrated exceptional capabilities, but their deployment in safety-critical environments is significantly hindered by their tendency to hallucinate and generate factually unsupported outputs. Existing detection methods often rely on fallible LLM evaluators, which cannot guarantee the reliability of the evaluating mechanism itself and leads to an infinite regress of trust. To address this, this project investigates the dynamics of hallucination propagation and self-correction in multi-agent LLM collaboration. We conduct an empirical study of opinion formation across 50-node Erdős-Rényi random graphs to demonstrate how network topology influences multi-round discussions among architecturally diverse LLM agents (Llama 3.2, Qwen 2.5, and Mistral). To ensure reliable accountability and rigorous ground-truth validation without relying solely on automated LLM judges, we integrate a Solidity smart contract-based audit and reputation system. This blockchain infrastructure provides tamper-resistant, penalty-weighted accountability for each agent, producing quantitative reliability and trust scores from immutable on-chain data recorded on a local Ethereum test chain. Experimental results demonstrate that denser network topologies accelerate convergence toward factual accuracy, and that the behavioral signals emerging from topology-constrained discussions carry substantial information about agent reliability independent of any automated detection mechanism.**

***Index Terms*—AI accountability, Blockchain Audit Trail, Hallucination Detection, Large Language Models**

## I. Introduction

Large Language Models (LLMs) have demonstrated exceptional capabilities across diverse tasks, but their deployment in safety-critical domains is severely hindered by the problem of hallucination [1]. Hallucinations occur when models confidently generate factually incorrect or logically unsupported content [2]. This problem is compounded by phenomena such as "hallucination snowballing", where models over-commit to their initial mistakes and fabricate further claims to justify them [3]. While various detection strategies exist, ranging from consistency-based sampling methods like SelfCheckGPT [4] to LLM-as-a-judge frameworks,they suffer from a critical structural limitation. Specifically, when an LLM evaluates another model's output, it cannot guarantee the reliability of its own judgment, leading to an infinite regress of trust rather than resolving the uncertainty [5].

To surpass the limitations of single-model reasoning, recent advancements have shifted towards multi-agent collaboration frameworks, inspired by the "Society of Mind" concept [6], [7]. By allowing multiple LLMs to propose, debate, and critique responses over several rounds, these systems can significantly enhance factual accuracy and reasoning capabilities [6]. However, decentralized multi-agent networks introduce new security vulnerabilities and operational challenges [8]. A confidently incorrect or adversarial agent can exploit the collaborative debate process to persuade benign agents to accept fabricated claims [9]. As a result, a single hallucinating agent can trigger a cascade of misinformation that compromises the collective consensus of the entire network [10].

The resilience of a multi-agent system against such contagious hallucinations is heavily influenced by its underlying communication topology [11]. While prior research has examined deterministic structures such as linear, flat, or hierarchical networks [11], simulating realistic and unpredictable interactions requires more complex topological models. This project addresses this gap by investigating opinion formation and hallucination dynamics across 50-node Erdős-Rényi random graphs with varying levels of connectivity [5]. By examining these stochastic networks, we can quantify how architectural diversity among agents (e.g., Llama 3.2, Qwen 2.5, Mistral) and specific graph densities govern whether fabricated claims propagate or self-correct during multi-round discussions [5].

To bypass the recursive trust problem inherent in automated LLM evaluation, this study relies on behavioural observation and rigorous manual ground-truth validation rather than explicit detection prompting [5]. Furthermore, to establish reliable accountability within the multi-agent network, we introduce a Solidity smart contract-based audit and reputation

system [5]. This blockchain infrastructure provides a tamper-resistant, penalty-weighted scoring mechanism that records every pipeline execution as immutable on-chain data [5]. By tracking system alignment through quantitative metrics like Shannon Entropy decay, this research provides a comprehensive framework to understand how multi-agent collaboration, network topology, and verifiable cryptographic penalties interact to isolate faulty agents and ensure a robust convergence toward factual accuracy [5].

## II. Literature Review

The rapid advancement of artificial intelligence has led to highly capable systems across various modalities; however, the deployment of Large Language Models (LLMs) introduces significant challenges in reliability, factuality, and security. A primary limitation of generative language models is their tendency to hallucinate, generating plausible but fabricated information [1]. A major driver of this issue is "hallucination snowballing," where models over-commit to their early mistakes and fabricate further claims to justify them over continuous generation [3]. Models also frequently generate self-contradictory statements within the same context, indicating a fundamental breakdown in logical consistency [12]. To address this, researchers have developed zero-resource methods like SelfCheckGPT to measure informational consistency across stochastic samples [4], as well as active detection strategies that calculate token-level uncertainty to spot potential hallucinations during text generation [13].

Once detected, mitigating these hallucinations traditionally involves augmenting models with external knowledge bases or validation loops. Systems like the RARR framework retroactively research and edit text to ensure it aligns with retrieved evidence [14], while the LLM-AUGMENTER system iteratively revises prompts using automated utility feedback [15]. Alternatively, LLMs can be forced to explicitly abstain from answering when knowledge gaps are detected via multi-model collaboration [16]. However, to rigorously evaluate the effectiveness of these mitigation strategies across iterative conversational rounds, establishing a systematic ground truth validation framework is essential. By continuously logging and comparing multi-round LLM outputs against a verified ground truth, developers can precisely track how fabricated facts evolve, measure the exact rate of hallucinations, and ensure that iterative evaluations actively drive the system toward factual accuracy rather than just superficial consensus [5], [13], [15].

This need for rigorous validation and iterative refinement naturally aligns with the shift toward multi-agent systems, which are designed to surpass the reasoning capabilities of a single LLM. By allowing multiple LLM instances—often utilizing varied architectures—to propose and evaluate their individual reasoning processes over several rounds, systems can reliably debate and converge on more accurate answers [6], [17]. The efficacy of this multi-agent debate is heavily influenced by the underlying communication topology. While studies often structure these networks using complete mesh, star, or ring formations, the introduction of random graph topologies provides a critical mechanism for simulating unpredictable, real-world interactions [5], [11]. Analyzing how agents share answers and evaluate peer responses across these diverse, randomized graph structures helps determine which network configurations are most resilient to error propagation, allowing the system to isolate faulty agents and facilitate the correction of faulty reasoning [11].

As multi-agent networks become more complex, the primary security risk shifts from isolated prompt injections to systemic vulnerabilities, particularly concerning hallucination-based attacks and the contagious spread of misinformation. In a collaborative environment, a hallucination generated by one agent can inadvertently act as an adversarial payload, persuading benign peer agents to deviate from their objectives and adopt incorrect facts. Studies investigating multi-agent collaboration attacks demonstrate that a single adversarial or confidently hallucinating agent can easily disrupt debate networks, causing errors to cascade through the entire system [9]. Furthermore, phenomena like "infectious jailbreaks" highlight how hallucinated or malicious inputs injected into one agent's memory can autonomously spread, infecting the reasoning processes of connected agents without any further external intervention [8], [18]. Understanding the dynamics of these hallucination-induced systemic failures is critical for developing resilient network topologies that can contain, identify, and neutralize fabricated information before it compromises the collective output of the agent network.

## III. Threat Model

Large Language Models deployed in multi-agent pipelines introduce compounded reliability risks: a single undetected hallucination can propagate through downstream reasoning steps, corrupting the final output in ways that are difficult to attribute or audit. This project addresses the problem of hallucination detection in a secure multi-entity LLM system, where correctness must be both computationally verified and cryptographically auditable.

The system operates under the following threat model. The principal assets requiring protection are the correctness of hallucination detection verdicts, the integrity of inter-agent scoring, and the immutability of the blockchain audit trail.

### A. Adversarial Threats

- *Direct User Input Injection*
  The prompts provided by the user in the *run_pipeline* function is sent directly to the *generate* method of each of the nodes without being verified or sanitised, meaning that an attacker can include instructions on adversarial overrides in the input. Since the LLMs treat the entire input as continuous natural language, injecting commands can override the intended system behaviour without requiring any code access. The attack is adversarial and its impact is further increased by the broadcast mechanism since the compromised input is shared among all nodes at

the very first round, ensuring every subsequent evaluation round propagates from a poisoned baseline.

- *Result Tampering*
  The *ground_truth* parameter is an unvalidated string passed into *run_pipeline* function and stored in the output CSV, which analysts use as the reference baseline for manually labelling hallucinations. Three attack forms exist:
  - *Factual Falsification*- An attacker injects an incorrect *ground_truth* value, which forces the analysts to compare all responses of the nodes against a false base. Proper responses become hallucinations and genuine hallucinations become clean and the results of detection are inverted.
  - *Selective Label Editing*- An attacker with post-run CSV access directly edits the *Hallucination* column as the file has no hash, signature, or audit trail, a single changed cell is completely undetectable.
  - *User Negligence*- In addition to intentional attacks, incorrect ground truth values may also be introduced through accidental user errors. For example, an analyst may unintentionally input an incorrect reference answer, misunderstand the correct response, or mistakenly edit hallucination labels during manual review. Since the system currently lacks automated verification or integrity checks, such mistakes can silently corrupt the evaluation results.

  This attack is purely adversarial because no legitimate user has reason to supply an incorrect ground truth or alter a label. It requires deliberate intent and insider knowledge of the pipeline's structure. The absence of any tamper-evident mechanism on the output CSV makes all three forms silent and undetectable by default.
- *Prompt Family Switching*
  The system also allows two immediate families that control the various evaluation logic: Family A commands nodes to independently re-check their responses whereas Family B combines neighbour responses to produce a combined response. The attacker who can access the pipeline can cause the prompt family to degrade hallucination detection, compelling Family B to dilute individual errors in the consensus noise, or compelling Family A to avoid stable convergence by causing uncoordinated re-evaluation.
- *Colluding Adversarial Agents*
  This threat model assumes that multiple agents are compromised and operate under coordinated adversarial control. These agents collaborate to manipulate the collective reasoning process and influence the final system output. By coordinating their responses, adversarial agents can increase their influence within the network and take over the consensus mechanism.

### B. Non Adversarial Attacks

- *Iterative Error Propagation*
  This threat occurs when an incorrect response generated during early reasoning stages spreads and intensifies across subsequent communication rounds within the multi-agent network. Because agents update their responses based on previously shared outputs from neighbouring nodes, an initial hallucinated answer or reasoning mistake can gradually influence other agents. As the system continues its iterative evaluation process, the incorrect information may propagate through the network and become embedded in later reasoning updates. Over time, this process can lead to widespread error amplification, where multiple agents converge on an incorrect conclusion despite the presence of correct information in earlier rounds.
- *Consensus Bias*
  Consensus bias arises when agents tend to adopt responses that appear frequently among neighbouring nodes. Even when correct information is present, the repeated incorrect response can bias agents toward accepting it, causing the system to agree on an inaccurate result.
- *Context Contamination*
  During multi-agent communication, responses from other agents are incorporated into the context of each agent's prompt. If this information contains errors or misleading reasoning, it can contaminate the reasoning process and lead agents to produce incorrect outputs.
- *Convergence Failure*
  In some cases, agents may fail to reach a stable consensus even after multiple rounds of communication. Repeated revisions based on conflicting responses can cause conflicting outputs, preventing the system from converging to a reliable solution.

## IV. Security Policy

The following security requirements govern this system, scoped to the implemented architecture and sufficient to protect against each threat in Section III.

### A. Independent Initial Reasoning

Each agent must generate its initial response independently before receiving information from neighbouring nodes. This ensures that early reasoning stages remain unaffected by external influence and provides a baseline for later comparison. Maintaining independent initial responses enables the system to detect when subsequent updates are the result of propagation effects rather than original reasoning.

### B. Cryptographic Accountability and Auditability

To protect the immutability of the audit trail, the following controls are mandated:

- *Verification Finality*
  The system does accept a hallucination verdict until a transaction has been successfully mined on the NodePenaltySystem smart contract. In every transaction, one should have nodeId, modelType, and strikeCount.

- *Identity Binding*
  Each agent taking part (i.e. Node 1, Node 2, Node 3) is cryptographically identifiable to a distinct address over the Ganache local blockchain in order to discourage identity spoofing or Sybil-based consensus attacks.

### C. Cryptographic File Integrity

To ensure the integrity and authenticity of pipeline output data, the system implements cryptographic integrity verification for all generated files. Immediately after each pipeline execution, a secure hash value such as SHA-256 is computed for every output, including CSV files containing node responses, hallucination labels, and audit records. The generated hash is stored separately and used as a reference for future verification. Before any further analysis or processing, the system recomputes the hash of the file and compares it with the stored value to detect any unauthorized modifications. Any mismatch indicates potential tampering, ensuring that the audit data remains reliable, traceable, and resistant to undetected manipulation.

### D. Data Analyst Accountability

To reduce risks arising from user negligence and unauthorised modification of evaluation outputs, the system enforces controlled access during the manual review process. Analysts are responsible for reviewing pipeline outputs and assigning hallucination labels in the generated CSV files. Since these labels influence subsequent analysis and node reputation scoring, analysts are permitted only to review outputs and update hallucination labels. Access to system configuration, pipeline parameters, and blockchain penalty mechanisms remains restricted to preserve the integrity and reliability of the overall system.

## V. System Design

### A. Design Rationale

Current hallucination detection methods including consistency-based approaches such as SelfCheckGPT, NLI-based entailment scoring, and LLM-as-a-judge frameworks share a fundamental architectural limitation: they cannot verify the reliability of the evaluating mechanism itself. When an LLM evaluates another LLM's output and the evaluator itself hallucinates, the detection verdict is compromised. Adding another layer of judges produces an infinite regress rather than resolving the uncertainty. This recursive trust problem is not a fixable implementation issue but a structural property of any system that uses LLMs to evaluate LLMs. This work takes an alternative approach: studying the *dynamics* of how hallucinated claims propagate and self-correct when agents interact on constrained network topologies. The hypothesis is that behavioural signals emerging from multi-round peer interaction carry information about agent reliability that is independent of any automated detection mechanism. This connects to the broader study of opinion formation dynamics on random networks and recent work on LLM-based agent simulation, while applying these concepts to a novel problem: hallucination identification through behavioural observation rather than content-level evaluation.

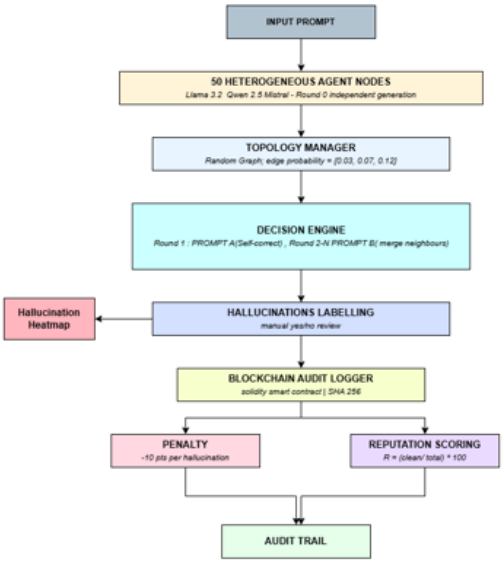


Fig. 1: *High-level design of the proposed multi-agent hallucination analysis system.*

### B. Multi-Agent Opinion Formation on Stochastic Networks

The empirical framework consists of five components: a node layer, a topology layer, a discussion engine and ground truth validation. No component performs explicit hallucination detection; instead, the system creates conditions for behavioural signals to emerge naturally from multi-round peer interaction.

*1) Node Layer:* Fifty stateful agents, each assigned one of three open-weight model architectures (Llama 3.2 3B, Qwen 2.5 7B, Mistral 7B), are instantiated as `Node` objects. Each node maintains a persistent `current_answer` attribute that carries state across discussion rounds. Model assignment follows a balanced rotation to ensure approximately equal representation in the giant component. The choice of three architecturally distinct models ensures that hallucination patterns differ across agents: different training corpora and model sizes produce different failure modes. This heterogeneity is essential for studying whether exposure to architecturally diverse peers facilitates hallucination correction. No agent is informed that it is being observed for hallucination; the system prompt contains only the node's identifier.

*2) Topology Layer:* Agents are connected via Erdős-Rényi random graphs $G(n, p)$ with $n = 50$ nodes, where the edge probability $p$ controls network density. The Erdős-Rényi model is chosen for its theoretical tractability: the percolation threshold at $c = p(n-1) = 1$ provides a principled boundary between fragmented and connected regimes. Three edge probabilities are tested:

- $p = 0.03$ ($c \approx 1.47$): sparse, near percolation threshold

- $p = 0.07$ ($c \approx 3.43$): moderate connectivity
- $p = 0.12$ ($c \approx 5.88$): dense, highly connected

For each graph instance, the giant component is extracted using NetworkX and isolated nodes are excluded. This ensures all participating agents can communicate, preventing trivially disconnected subgroups from confounding results.

*3) Discussion Engine:* In Round 0, each agent answers the question independently with no access to other agents' responses, enforcing generation independence. From Round 1 onward, each agent receives its topological neighbours' current answers and is prompted to merge them into a single improved answer. Critically, the prompt does not instruct agents to identify errors, evaluate truthfulness, or detect hallucination it requests only synthesis. The discussion proceeds for 50 rounds, with answers recorded at Round 0 and every 5th round. This sampling strategy was validated through a 50 round pilot that confirmed convergence is typically established within this window. Opinion dynamics emerge naturally: agents with correct answers tend to hold firm, agents with hallucinated answers tend to adopt the majority position, and in some cases a confidently hallucinated agent persuades correct agents to adopt a false claim.

*4) Ground Truth Validation:* After each run, recorded answers are manually annotated against known ground truth by the full research team, producing boolean hallucination labels per agent per sampled round. Manual validation is deliberately chosen over automated detection to avoid introducing the recursive trust problem that motivates this work.

### C. Blockchain Audit and Reputation Infrastructure

The blockchain infrastructure addresses two assessed dimensions simultaneously: security through tamper-evident audit logging and hash-based integrity assurance, and resilience through reputation-based quality scoring of agent outputs.

*1) Security Properties Addressed:* The smart contract directly addresses the following security requirements:

Integrity and controlled authenticity. Every agent's response is committed to the blockchain as an immutable `AuditEntry` struct containing round number, hashed node identifier, hashed model name, hashed answer, hashed ground truth, hallucination flag, penalty applied, and block timestamp. Once a `recordAudit()` transaction is mined, it cannot be altered or deleted without invalidating the chain state.

Access control. The `recordAudit()` function is restricted to the deployer account via the `onlyOwner` modifier, preventing unauthorised writes to the audit trail.

Audit logging. Every pipeline execution commits a sequence of immutable records. The contract exposes `getAuditCount()` and `getAuditEntry()` view functions for independent verification. The `AuditRecorded` event enables efficient off-chain monitoring. This directly satisfies the audit log criterion described in the module brief.

*2) Resilience Properties Addressed:* The blockchain reputation system implements a scoring mechanism that assesses and ranks the quality of agent outputs:

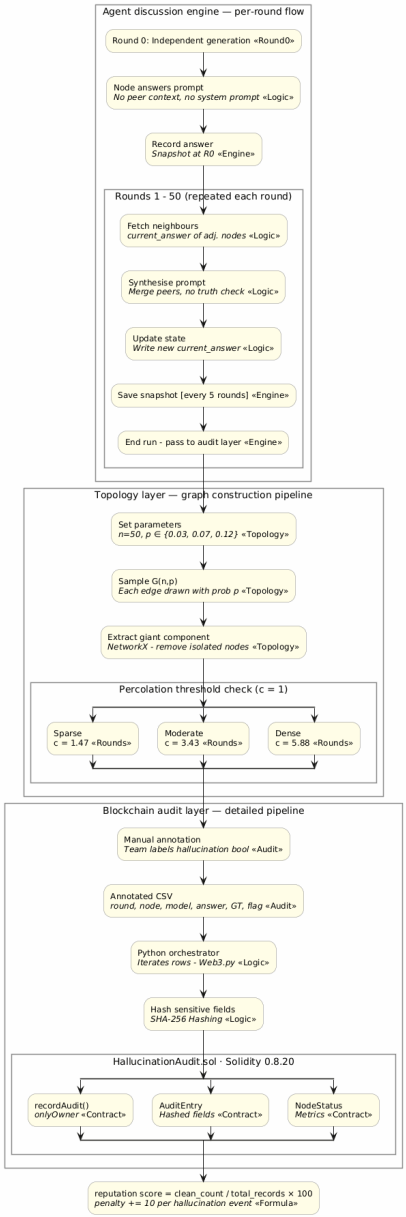


Fig. 2: *Architectural workflow of the multi-agent discussion system, illustrating the per-round generation logic, the $G(n, p)$ topology construction layer, and the subsequent blockchain-based audit pipeline for reputation scoring.*

Reputation-based scoring. The `NodeStatus` struct maintains cumulative statistics per node: total penalties, hallucination count, and clean count. A reputation score is derived as a percentage of clean outputs over total records.

$$\text{reputation score} = \frac{\text{clean_count}}{\text{total records}} \times 100 \tag{1}$$

Penalty mechanism. Each hallucination event incurs a fixed penalty of 10 units (defined in the orchestration layer and applied only when hallucination is true), creating a cumulative cost signal. Agents that hallucinate more frequently accumulate higher penalties.

System-level resilience. The limitation of unreliable agents is supported by the broader multi-agent system design rather than being directly implemented within the smart contract.

*3) Smart Contract Implementation:* The Solidity smart contract (`HallucinationAudit.sol`, version 0.8.20) is deployed on a local Ethereum test chain using `EthereumTesterProvider`. The contract is compiled using `py-solc-x` and deployed via Web3.py. A Python orchestration script iterates over each row of the manually annotated CSV, hashes sensitive fields (node, model, answer, ground truth), and calls `recordAudit()` per entry.

## VI. Implementation

All experiments are executed on Google Colab with GPU acceleration. Ollama is installed at runtime and serves models locally via the REST API. Three open-weight models are used: Llama 3.2 (3B parameters), Qwen 2.5 (7B parameters), and Mistral (7B parameters). Each agent is a Python `Node` class with three attributes: unique node identifier, assigned model name, and persistent `current_answer` string. The `generate()` method produces the Round 0 answer. The `evaluate()` method, called from Round 1 onward, constructs the merge prompt from all visible neighbours' answers and updates `current_answer`. Four topology generators are implemented: complete mesh, star, ring, and random graph. For random graphs, `get_largest_component()` extracts the giant component using NetworkX. The `run_pipeline()` function orchestrates the full sequence. Answers are recorded to CSV at Round 0 and every 5th round. Each row contains round number, node identifier, model name, answer text, ground truth, and hallucination label column for manual annotation. The Solidity source is compiled using `py-solc-x` (solc 0.8.20) and deployed to a local Ethereum test chain via `Web3(EthereumTesterProvider())`. The orchestration script iterates over annotated CSV rows, calling `recordAudit()` per entry. Heatmaps use Matplotlib with a binary colour map (green = correct, red = hallucinated). Network topology visualisations use NetworkX's spring layout with nodes coloured by model architecture.

## VII. Methodology

This is a quantitative repeated-measures experiment focused exclusively on the current implementation. The study uses 50 agents across three category of ground-truth questions (Factual, Mathematical puzzles, Decoding).

Graphs: Erdős-Rényi random with three densities. Procedure: Round 0 independent inference $\rightarrow$ neighbour exchange $\rightarrow$ majority merge with model weighting. 21 experimental runs in total (one run = one question on one density). The first experiment was conducted for 100 rounds but due to computational power restrictions all subsequent experiments were run for 50 rounds.

Metrics :

$$\text{Hallucination Rate} = \frac{\text{hallucinated nodes}}{50}$$

Shannon entropy (binary):

$$H_b = -p\log_2(p) - (1-p)\log_2(1-p) \tag{2}$$

Answer-distribution entropy (multi-class):

$$H_a = -\sum p_i \log_2(p_i) \tag{3}$$

Lower values in later rounds prove successful consensus and reduced spreading. All data exported to CSV; blockchain trust scores recorded and penalties applied for hallucinating with blockchain audit trail.

$$H(t) = -\sum_{i=1}^{n} P(x_i)\log_2 P(x_i) \tag{4}$$

where $P(x_i)$ is the frequency of a unique response cluster $x_i$ appearing across the nodes. High $H$ at Round 0 represents baseline stochastic independence, while the rate of decay ($\Delta H$) evaluates convergence efficiency.

## VIII. Evaluation Methodology

The evaluation framework is designed to move beyond subjective observation of model outputs, utilizing mathematical and structural benchmarks to quantify the stability and convergence dynamics of the 50-node multi-agent network. The study evaluates the system across three primary dimensions:

### A. Quantitative Metric: Shannon Entropy (H)

- Benchmark: Shannon Entropy is the primary metric for assessing system-wide alignment.
- Application: The network's "disorder" is evaluated by treating the aggregate of agent responses as a probability distribution.
- Definition: For each communication round t, entropy is calculated as:

$$H(t) = -\sum_{i=1}^{n} P(x_i)\log_2 P(x_i) \tag{5}$$

  where $P(x_i)$ is the frequency of a unique response cluster $x_i$ appearing across the nodes.
- Criteria: Entropy serves as a proxy for disagreement. High $(H)$ at Round 0 represents baseline stochastic independence, while the rate of decay $(\Delta H)$ across subsequent rounds evaluates convergence efficiency.

### B. Structural Benchmark: Topological Efficiency

- Benchmark: We evaluate the influence of architecture by analysing how the 50-node graph facilitates or hinders information flow.
- Variable: The study utilizes a random graph topology with stochastic edge probabilities (0.03, 0.07, and 0.12) to ensure a non-trivial communication environment.
- Criteria: We measure Consensus Density, the ability of the network to prevent "information siloing". The focus is on whether the topology provides enough connectivity for three heterogeneous model types to reach a unified state.

### C. Statistical Reliability and Variance Analysis

A critical component of this evaluation is the aggregation of 21 independent trials. To ensure the results are a robust property of the architecture rather than specific random initializations, we analyse the following:

- Mean Entropy ($\mu H$): The average entropy across all 21 runs for each round t.
- Standard Deviation ($\sigma$): The variance in entropy decay across all runs, used to evaluate the system's "Corrective Pressure"
- Reliability Proof: A narrowing standard deviation over time serves as evaluative proof that convergence is a robust property of the architecture rather than a result of specific random initializations
- Normal Distribution Analysis: A Gaussian analysis is applied to final-round entropy to quantify the mathematical certainty of convergence.
- Reliability Interval ($1\sigma$): We define the shaded $1\sigma$ region (representing a 68.2% probability) as the high-confidence range for network performance.

### D. Blockchain-Based Audit and Penalty Evaluation

The blockchain layer is evaluated across two dimensions that mirror the infrastructure design: *security*, assessed through the integrity and correctness of the tamper-evident audit trail, and *resilience*, assessed through the discriminability and proportionality of the reputation-based penalty mechanism.

*Security: Audit Integrity:* Each agent response is committed on-chain as an immutable `AuditEntry` via `recordAudit()`, restricted to the deployer account via `onlyOwner`. The row-level audit hash is

$$H_{\text{row}} = \kappa(h_{\text{node}} \parallel h_{\text{model}} \parallel h_{\text{ans}} \parallel h_{\text{gt}}) \tag{6}$$

where $\kappa(\cdot)$ denotes `keccak256` and $h_x = \text{SHA-256}(x)$ for each field $x$. Evaluation criteria are: (i) every pipeline row produces a unique, deterministic $H_{\text{row}}$; (ii) `getAuditCount()` returns $n_k \times T$ entries per run, where $n_k$ is the active node count and $T = 11$ the sampled rounds; and (iii) no record is writable by an account other than the deployer.

*Resilience: Reputation Scoring:* The `NodeStatus` struct maintains per-node cumulative statistics. The reputation score and cumulative penalty for node $i$ are:

$$R_i = \frac{c_i}{r_i} \times 100, \qquad P_i = \delta \cdot h_i \tag{7}$$

where $c_i$ is the clean record count, $r_i$ the total records, $h_i$ the hallucination count, and $\delta = 10$ the fixed penalty defined in the orchestration layer and applied only when hallucination is true. Evaluation criteria are: (i) $P_i$ scales linearly with $h_i$; (ii) $R_i$ spans $[0, 100]$, distinguishing full convergence ($R_i = 100$), partial convergence ($0 < R_i < 100$), and total failure ($R_i = 0$); and (iii) the mean reputation per edge probability across all $K = 21$ runs,

$$\bar{R}(p) = \frac{1}{N_p} \sum_{k=1}^{K} \sum_{i=1}^{n_k} R_i^{(k)}, \qquad N_p = \sum_{k=1}^{K} n_k \tag{8}$$

is expected to increase monotonically with $p$, providing empirical confirmation that greater topological connectivity is associated with a more reliable agent population.

## IX. Experimental Results

### A. Quantitative Analysis of 50-Node Alignment

The objective of this analysis is to characterize the transition of a large-scale, 50-node multi-agent system from initial stochastic divergence to collective alignment. As illustrated in Fig. 3, the system demonstrates a clear "micro-to-macro" transition across 21 independent trials.

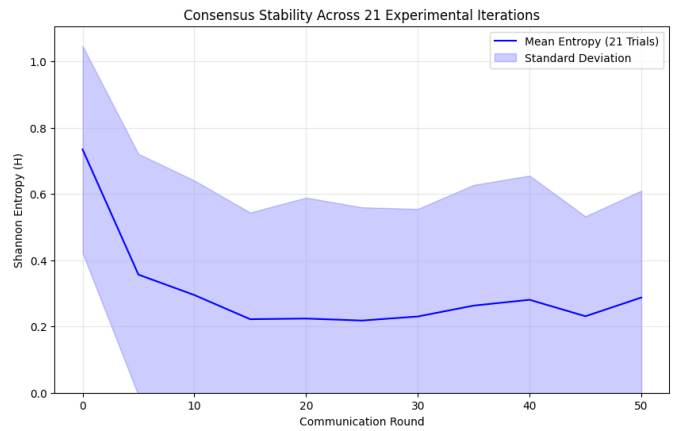


Fig. 3: *Mean Shannon Entropy decay and standard deviation ($\sigma$) across 21 trials. High initial entropy ($\approx 0.74$) reflects the baseline disorder of uncoordinated models.*

The convergence process is defined by three primary phases:

- Initial Stochasticity (Round 0): High mean entropy ($\approx 0.74$) and significant standard deviation reflect the baseline "disorder" of 50 uncoordinated models.
- Phase of Rapid Convergence (Rounds 1–15): Asharp decline in entropy is observed upon activation of the communication protocol. By Round 15, mean entropy drops significantly to $\approx 0.22$, indicating a powerful alignment mechanism.
- Steady-State Consensus (Rounds 15–50): Following the initial convergence, the system enters a stable plateau where entropy remains significantly lower than the baseline. However, as seen in Fig. 1, a slight increase in mean entropy and standard deviation occurs between Rounds 30 and 50. This suggests that while the 50-node network reliably reaches a singular state, it maintains a degree of "Consensus Drift." This late-round volatility indicates that even in a stable topology, cumulative iterations can introduce minor stochastic fluctuations that prevent a permanent 0.0 entropy state.

### B. Topological Influence and Model Heterogeneity

By employing a 50-node topology across three distinct LLM architectures with varying edge probabilities (0.03, 0.07, 0.12), the study ensured that information was shared effectively across the collective.

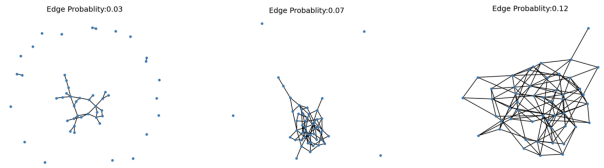


Fig. 4: *Visualization of the 50-node random graph topology with different edge probabilities (0.03, 0.07, and 0.12).*

Our observations suggest that the *architecture of interaction* is the dominant factor in system stability. Even when individual nodes provided divergent data, the collective structural pressure of the 50-node network successfully filtered out these stochastic errors.

### C. Probability Analysis and Reliability

The Aggregated Hallucination Probability Map (Fig.6) provides a granular, node-level visualization of the "micro-to-macro" transition. In Round 0, the map displays high probability error clusters, which reflect the baseline stochastic independence of the uncoordinated 50-node network. As communication rounds progress, the probability gradient across the nodes shifts significantly toward a stable "green" state. This visual transition confirms that the iterative structural pressure of the topology successfully filters out individual agent hallucinations.

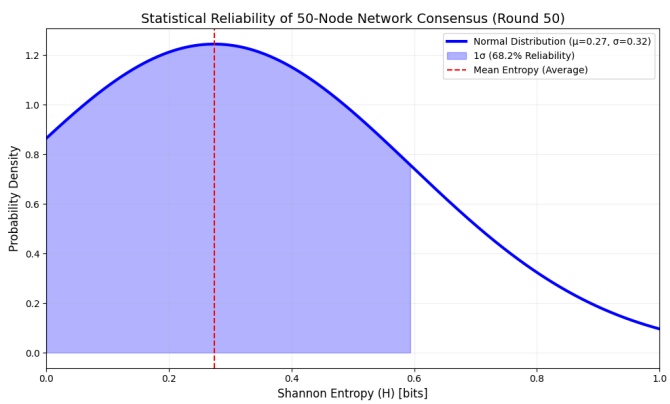


Fig. 5: *Probability Distribution of 50-Node Network Consensus at Round 50. The peak at $\mu = 0.27$ bits and the shaded $1\sigma$ reliability interval demonstrate the consistent suppression of hallucinations across 21 independent trials.*

To quantify the statistical certainty of this transition, a Normal Distribution (Gaussian) analysis was conducted on the final-round entropy results across all 21 trials (Fig.5). This analysis evaluates whether the observed convergence is a robust property of the architecture rather than a result of specific random initializations

- Mean Final Entropy ($\mu = 0.27$): The peak of the distribution confirms that the most probable state for the 50-node collective is one of high logical alignment and low informational disorder.
- Reliability Interval ($1\sigma$): The shaded $1\sigma$ region (representing a 68.2% probability) illustrates the high-confidence range for network performance. The clustering of results between $H = 0.0$ and $H = 0.59$ proves that the "corrective pressure" drives the system toward consensus with high mathematical certainty.
- Evidence of Consensus Drift: The standard deviation ($\sigma = 0.32$) provides a quantitative measure of the terminal volatility observed in later rounds. While the mean remains low, the width of the bell curve captures the instances of "Consensus Drift," where cumulative iterations introduce minor stochastic fluctuations that prevent a permanent 0.0 entropy state.
- By synthesizing the "micro" node-level data from the heatmap with the "macro" probability density of the bell curve, we demonstrate that reliability in multi-agent LLM systems can be derived from the mathematical stability of a convergent collective

### D. Blockchain-Based Audit and Reputation Scoring

To extend the system's integrity guarantees beyond statistical inference, a blockchain-based audit layer was integrated to enforce accountability at the individual agent level. Each agent response was hashed and recorded on-chain alongside its hallucination flag and ground-truth comparison result. Smart contract execution was triggered automatically whenever a hallucination was detected (flagged as TRUE in the manually verified CSV output), applying a configurable penalty to the responsible node. This mechanism transforms the hallucination signal from a passive diagnostic into an active enforcement event. The audit CSV output captures the full per-record ledger including node hash, model hash, answer hash, ground truth hash, hallucination flag, penalty applied, timestamp, reputation score, and an immutable audit row hash.

These results from TABLE I demonstrate that the hallucination heatmap derived from the multi-agent network's historical outputs can serve as a direct input to the blockchain reputation engine. Nodes with consistently low hallucination rates accumulate higher reputation scores on-chain, while nodes with recurrent errors incur escalating penalties. This closed-loop mechanism bridges the probabilistic inference of the collective network with a tamper evident ledger, providing an auditable basis for trust assignment in multi-agent AI deployments.

### E. Penalty Engine Results

The penalty engine was triggered automatically by the smart contract whenever a hallucination flag (TRUE) was submitted via `audit_record()`. A fixed deduction of 10 reputation points was applied per occurrence. Table III summarises the penalty totals, hallucination rates, and mean reputation scores aggregated across all 21 experimental runs, grouped by edge probability.

The penalty results confirm that the smart contract fires correctly and consistently across all network conditions, and that topological connectivity is a measurable determinant of penalty severity. At $p$=0.03, the 7 runs across this edge

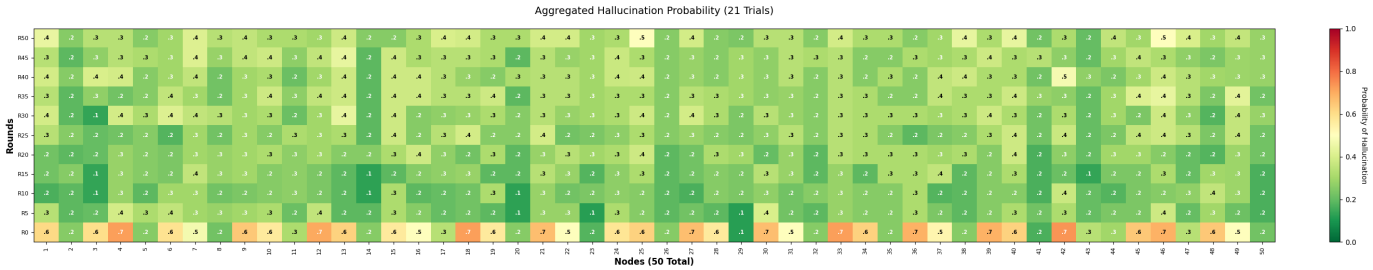


Fig. 6: *Aggregated Hallucination Probability Map showing the transition from high-probability error clusters (orange) to stable consensus (green) across 21 trials.*

TABLE I: Blockchain Audit Ledger - Factual Prompt 1 ($p$=0.03, 19 nodes, 11 sampled rounds, 209 total records). Each row is an immutable on-chain `AuditRecord`. Hashes truncated to 8 hex chars; full values in `blockchain_audit_output.csv`. *Hallucinated* = True/False; Pen. = penalty points deducted; Rep. = cumulative reputation score (0–100). Row 4 triggered the smart-contract penalty engine.

| Node Idx | Rd | Node Hash | Model Hash | Answer Hash | GT Hash | Hallucinated | Pen. | Timestamp | Rep. | Audit Row Hash |
|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | `53e9fcaf...` | `3901fbe4...` | `e5ca3fd2...` | `98b53185...` | False | 0 | 1773753174 | 100.00 | `92c1abed...` |
| 1 | 0 | `e5a2cb5a...` | `3e780275...` | `4feccf85...` | `98b53185...` | False | 0 | 1773753175 | 90.91 | `10ff0aea...` |
| 2 | 0 | `8c101ca0...` | `3901fbe4...` | `76a9e85b...` | `98b53185...` | False | 0 | 1773753176 | 100.00 | `3b85eaa9...` |
| 3 | 0 | `606ae0ca...` | `3e780275...` | `e2b39f22...` | `98b53185...` | False | 0 | 1773753177 | 100.00 | `b8dc3211...` |
| 4 | 0 | `980abe26...` | `37d9093e...` | `7dda7091...` | `98b53185...` | **True** | **10** | 1773753178 | 81.82 | `208bfcba...` |
| $\cdots$ *records 5–207 omitted; 16 hallucinations total across* 19 *nodes* $\times$ 11 *rounds* $=$ 209 *records* $\cdots$ | | | | | | | | | | |
| 208 | 50 | `9dd47d9c...` | `3e780275...` | `62440321...` | `98b53185...` | False | 0 | 1773753382 | 100.00 | `c5adf412...` |

**Note:** *Rep.* = cumulative blockchain reputation score (0–100) at time of transaction. Ground-truth hash is identical across all 209 rows (same canonical answer string). SHA-256 file-level digest of the full CSV: `blockchain_audit_output.sha256.txt` (published alongside audit trail).

TABLE II: Node penalty summary aggregated across all 21 experimental runs, grouped by edge probability $p$. *Total Records* = total audit records submitted to the blockchain (nodes $\times$ sampled rounds). *Total Pen.* = cumulative smart-contract penalty deductions (10 pts per hallucination). *Hall. %* = overall hallucination rate. *Mean Rep.* = mean node reputation score (0–100).

| Edge $p$ | Total Records | Total Penalty | Hallu. % | Mean Rep. |
|---|---|---|---|---|
| 0.03 | 1,815 | 6,410 | 35.32 | 64.68 |
| 0.07 | 3,729 | 12,730 | 34.14 | 65.86 |
| 0.12 | 3,839 | 10,480 | 27.30 | 72.70 |

**Note:** *Each edge probability group aggregates 21 experimental runs. Higher edge probability ($p$=0.12) yields the lowest hallucination rate (27.30%) and highest mean reputation (72.70), confirming that greater topological connectivity improves collective accuracy.*

condition produced 1,815 total audit records, of which 641 were flagged as hallucinations a 35.32% overall hallucination rate accumulating 6,410 total penalty points and a mean node reputation of 64.68. At $p$=0.07, the larger active components yielded 3,729 records with 1,273 hallucinations (34.14%), producing the highest total penalty tally of 12,730 points and a mean reputation of 65.86. The elevated penalty total at $p$=0.07 relative to $p$=0.03 reflects the greater number of nodes participating across the 7 runs rather than a worsening per-node hallucination rate. At $p$=0.12, the network demonstrated the strongest overall performance: 3,839 records with 1,048 hallucinations yielded the lowest hallucination rate of 27.30%, a correspondingly reduced penalty total of 10,480 points, and the highest mean reputation of 72.70. The monotonic improvement in both hallucination rate and mean reputation as edge probability increases from 0.03 to 0.12 provides quantitative confirmation, via the blockchain ledger, that greater topological connectivity generates stronger corrective pressure and produces a more reliable node population over repeated experimental runs.

## X. Discussions and Extended Contribution

### A. Overall Discussion

The 50-node multi-agent network's behaviour varies significantly across prompt types and topological conditions. For the *Math Problem*, peer-driven corrective pressure was most effective, with mean node reputation scores of 96.23, 97.27, and 95.27 across the three edge probabilities the strongest results observed across the entire experiment. For the *Factual Prompt*, the network successfully suppressed initial hallucinations through consensus, with mean reputations consistently above 92 and improving with higher edge probability, confirming that greater connectivity improves collective accuracy on well-defined factual questions. The *decode prompt* category represents a key limitation of the current experimental setup. While convergence was observed from as early as rounds

1 indicating that the network did reach consensus the consensus itself was erroneous, resulting in a mean reputation of 0.00 across all edge probabilities. The shared decoding error propagated and stabilised rather than being corrected, which reflects a constraint of the open-source model tier employed. Notably, preliminary evaluations using the Gemini API demonstrated accurate decoding outputs; however, financial and computational constraints precluded its integration into the full experimental pipeline at scale.Future work with access to higher-capacity commercial models may resolve this limitation and extend the applicability of the penalty-based enforcement mechanism to more cognitively demanding task categories.

The blockchain audit layer demonstrated consistent value across all conditions. In successful runs, it produced per-node reputation scores that distinguish clean convergence from late-round drift a level of granularity that aggregate entropy metrics cannot provide. In failed runs, it generated a tamper-evident, cryptographically verifiable record of collective failure, with penalty totals and zero-reputation node counts quantifying the severity of breakdown. Its central contribution is not to alter network behaviour but to ensure that behaviour whether correct or catastrophically wrong is permanently and immutably recorded. A second significant limitation concerns the blockchain infrastructure used for the audit layer. Due to financial constraints, the project relied on a local Ethereum test chain provisioned via EthereumTesterProvider rather than a live public network such as the Ethereum mainnet or a funded testnet. While this was sufficient to demonstrate the functional correctness of the HallucinationAudit smart contract and the tamper-resistant audit trail, it foregoes the decentralisation and immutability guarantees that a production blockchain deployment would provide. On a real network, each transaction would be independently validated by distributed nodes, making post-hoc manipulation cryptographically infeasible. Deploying on the Ethereum mainnet or a production-grade testnet carries gas fees and ongoing infrastructure costs that were beyond the scope of this project. Future iterations should target a live deployment to fully realise the trust and accountability properties that blockchain-based audit trails are designed to offer.

### *B. Extended Contribution: Proposed Hallucination Detection Framework*

Building on the empirical findings, we propose a scalable three-layer hallucination detection architecture that eliminates the dependency on manual ground-truth validation. The architecture is complete at the component level; implementation and empirical evaluation constitute future work.

In this design, a single primary agent generates a response while nine verification agents, spanning five model architectures, independently answer the same question. The primary answer is then evaluated through three sequential methods:

- Method 1 - Cosine Similarity ($w_1 = 0.25$): Responses are encoded using `all-MiniLM-L6-v2` and pairwise similarities computed.
- Method 2 - NLI Contradiction ($w_2 = 0.35$): Logical relationships are classified using `cross-encoder/nli-deberta-v3-small`.
- Method 3 - Consensus Panel Judge ($w_3 = 0.40$): Every agent evaluates every other agent across four dimensions.

Method scores are combined via weighted adaptive fusion:

$$\mathcal{R} = w_1 R_1 + w_2 R_2 + w_3 R_3, \quad \sum_{j=1}^{3} w_j = 1 \tag{9}$$

The decision engine maps $\mathcal{R}$ to one of three outcomes: *accept*, *flag*, or *regenerate*. Cross-method characterisation yields eight flagging patterns, each corresponding to a distinct hallucination type a taxonomic approach absent from the reviewed literature that represents a novel design contribution.

Method 3 is subject to the recursive trust problem: agents evaluating agents may themselves hallucinate. Future work will integrate the per-node behavioural signals produced by the empirical study including blockchain reputation scores and per-round hallucination rates as an additional weighted input to the fusion layer, creating a hybrid system grounded in verified historical reliability.

## XI. Conclusion and Future Work

This study demonstrates that consensus is a quantifiable and repeatable property within a 50-node networked large language model (LLM) system. By utilizing mathematical and structural benchmarks, such as Shannon Entropy, to measure system-wide alignment, this research maps a clear transition from initial high-probability error clusters to a stable, low-probability consensus. Ultimately, iterative interactions among the agents generate an inherent "corrective pressure" within the network topology that successfully suppresses stochastic errors. The consistent stabilization of the 50-node system proves the feasibility of using multi-agent architectures to develop high-integrity AI protocols, deriving reliability from the mathematical stability of a convergent collective.

While this study successfully characterizes the transition from stochastic divergence to collective alignment in a 50-node network, several avenues for future research remain:

• Analysis of Consensus Drift: We intend to investigate the minor entropy fluctuations observed in terminal rounds (Rounds 35–50) to determine if the network is oscillating between near-identical consensus states or if cumulative semantic noise begins to degrade alignment over time.

• Automated Semantic Similarity Metrics: While the current study utilizes manual classification to identify hallucinations and agreement, future iterations will transition to automated semantic clustering. By utilizing vector embeddings and cosine similarity, we can replicate human-level logical assessment at scale, allowing for the evaluation of much larger networks (e.g., >500 nodes).

• Social Network Topology and Node Centrality: We plan to transition from simple random graphs to Social Network Graph models (e.g., Scale-Free or Small-World networks). This involves testing how Degree Centrality where specific

”hub” nodes have a higher number of connections affects the speed of the ”micro-to-macro” transition and whether these central nodes can act as anchors for faster network stability.

## References


[1] V. Rawte, A. Sheth, and A. Das. A survey of hallucination in “large” foundation models. *arXiv preprint arXiv:2309.05922*, 2023.

[2] J. Maynez, S. Narayan, B. Bohnet, and R. McDonald. On faithfulness and factuality in abstractive summarization. *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 2020.

[3] M. Zhang, O. Press, W. Merrill, A. Liu, and N. A. Smith. How language model hallucinations can snowball. *arXiv preprint arXiv:2305.13534*, 2023.

[4] P. Manakul, A. Liusie, and M. J. F. Gales. SelfCheckGPT: Zero-resource black-box hallucination detection for generative large language models. *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023.

[5] A. Jacob, A. Mani, C. George, R. Varunsegar, S. Patil, and S. Shetty. Dynamics of hallucination propagation in multi-agent LLM collaboration on graph topologies with penalty-based enforcement and a proposed automated detection framework. *Newcastle University*, 2024.

[6] Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, and I. Mordatch. Improving factuality and reasoning in language models through multiagent debate. *Proceedings of the Forty-first International Conference on Machine Learning*, 2023.

[7] M. Minsky. *Society of Mind*. Simon and Schuster, 1988.

[8] Z. Deng, Y. Guo, C. Han, W. Ma, J. Xiong, S. Wen, and Y. Xiang. AI agents under threat: A survey of key security challenges and future pathways. *ACM Computing Surveys*, 57(7), 2025.

[9] A. Amayuelas, X. Yang, A. Antoniades, W. Hua, L. Pan, and W. Wang. Multiagent collaboration attack: Investigating adversarial attacks in large language model collaborations via debate. *Findings of the Association for Computational Linguistics: EMNLP 2024*, 2024.

[10] Y. Pan, L. Pan, W. Chen, P. Nakov, M. Kan, and W. Y. Wang. On the risk of misinformation pollution with large language models. *arXiv preprint arXiv:2305.13661*, 2023.

[11] J. Huang, J. Zhou, T. Jin, X. Zhou, Z. Chen, W. Wang, Y. Yuan, M. R. Lyu, and M. Sap. On the resilience of LLM-based multi-agent collaboration with faulty agents. *Proceedings of the 42nd International Conference on Machine Learning*, 2025.

[12] N. Mündler, J. He, S. Jenko, and M. Vechev. Self-contradictory hallucinations of LLMs: Evaluation, detection and mitigation. *arXiv preprint arXiv:2305.15852*, 2023.

[13] N. Varshney, W. Yao, H. Zhang, J. Chen, and D. Yu. A stitch in time saves nine: Detecting and mitigating hallucinations of LLMs by validating low-confidence generation. *arXiv preprint arXiv:2307.03987*, 2023.

[14] L. Gao, Z. Dai, P. Pasupat, A. Chen, A. T. Chaganty, Y. Fan, V. Y. Zhao, N. Lao, H. Lee, D. C. Juan, and K. Guu. RARR: Researching and revising what language models say, using language models. *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, 2023.

[15] B. Peng, M. Galley, P. He, H. Cheng, Y. Xie, Y. Hu, Q. Huang, L. Liden, Z. Yu, W. Chen, and J. Gao. Check your facts and try again: Improving large language models with external knowledge and automated feedback. *arXiv preprint arXiv:2302.12813*, 2023.

[16] S. Feng, W. Shi, Y. Wang, W. Ding, V. Balachandran, and Y. Tsvetkov. Don’t hallucinate, abstain: Identifying LLM knowledge gaps via multi-LLM collaboration. *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024.

[17] Y. Dong, X. Jiang, Z. Jin, and G. Li. Self-collaboration code generation via ChatGPT. *ACM Transactions on Software Engineering and Methodology*, 33(7), 2024.

[18] X. Gu, X. Zheng, T. Pang, C. Du, Q. Liu, Y. Wang, J. Jiang, and M. Lin. Agent Smith: A single image can jailbreak one million multimodal LLM agents exponentially fast. *Proceedings of the Forty-first International Conference on Machine Learning*, 2024.

[19] Network Science Notes, “Random Graphs,” *Network Science Notes*, 2024. [Online]. Available: https://network-science-notes.github.io/chapters/09-random-graphs.html


## Appendix

### Appendix: Individual Contributions

Group contributions have been agreed among the group, and are fair and truthful. All six members contributed equally across all phases of the project, including literature review, system design, implementation, experimentation, evaluation, and report writing.

| **Member** | **Contribution (%)** |
|---|---|
| Alan Jacob | 16.67% |
| Apoorv Mani | 16.67% |
| Chris George | 16.67% |
| Ramya Varunsegar | 16.67% |
| Shreya Patil | 16.67% |
| Shreyas Shetty | 16.67% |
| **Total** | **100.00%** |

TABLE III: *Individual contributions of group members.*